

Producers-Consumer Problem
10 marks

5/5 answered

Your Code

```

1 import java.util.Scanner;
2 import java.util.InputMismatchException;
3 class InsufficientBalanceException extends Exception{
4     public InsufficientBalanceException(String message){
5         super(message);
6     }
7 }
8 class Main{
9     public static void main(String[] args){
10         Scanner scanner= new Scanner(System.in);
11         double balance=0;
12         double withdrawlAmount=0;
13         boolean isValidInput=false;
14         try{
15             if(scanner.hasNextLine()){
16                 String line=scanner.nextLine().trim();
17                 String[] parts=line.split("\\s+");
18
19                 if(parts.length<2){
20                     System.out.println("Invalid input");
21                 }
22                 else{
23                     try{
24                         balance=Double.parseDouble(parts[0]);
25                         withdrawlAmount=Double.parseDouble(parts[1]);
26                         isValidInput=true;
27                     }
28                     catch(NumberFormatException e){
29                         System.out.println("Invalid input");
30                     }
31                 }
32             }
33         } else{
34             System.out.println("Invalid input");

```

Input

10000 3000

Output



Online Food Delivery - Multiple
Threads and Execution Order
10 marks

Q5

Manufacturing Plant -
Producer-Consumer Problem
10 marks

5/5 answered

```

1 import java.util.Scanner;
2 public class HospitalPatientRegistration{
3     static class InvalidAgeException extends Exception{
4     }
5     static class InvalidTemperatureException extends Exception{
6     }
7     }
8     public static void main(String[] args){
9         Scanner sc=new Scanner(System.in);
10        try{
11            String name=sc.next();
12            int age=sc.nextInt();
13            double temperature = sc.nextDouble();
14            if(age<10){
15                throw new InvalidAgeException();
16            }
17            System.out.println("Success");
18        }
19        catch(InvalidAgeException e){
20            System.out.println("InvalidException");
21        }catch(Exception e){
22            System.out.println("Invalid Input");
23        }finally{
24            System.out.println("completed");
25        }
26    }
27 }

```

atm: 35.98.6

Output

```

Success
Completed

```

 Submitted

5/5 answered

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String[]products={"Laptop","phone","headphones"};
6         double[]prices={50000,30000,2000};
7         try{
8             int index=sc.nextInt();
9             int quantity=sc.nextInt();
10            if(quantity<=0){
11                throw new IllegalArgumentException("Invalid quantity");
12            }
13            double total=prices[index]*quantity;
14            System.out.println("Product: "+products[index]);
15            System.out.println("Total Cost: "+total);
16        }
17        catch(ArrayIndexOutOfBoundsException e){
18            System.out.println("Invalid product Index");
19        }
20        catch(java.util.InputMismatchException e){
21            System.out.println("Invalid numeric input");
22        }
23        catch(IllegalArgumentException e){
24            System.out.println("Qunatity must be greater than 0");
25        }
26        System.out.println("Transaction processing completed.");
27    }
28 }
```

1/2

Output

```
Product: phone
Total Cost: 40000.0
Transaction processing completed.
```

Submitted

5/5 answered

```

1 import java.util.Scanner;
2 public class OnlineFoodDelivery{
3     static boolean paymentDone=false;
4     public static void main(String[] args) throws Exception{
5         Scanner sc=new Scanner(System.in);
6         String order=sc.next();
7         String payment=sc.next();
8         String delivery=sc.next();
9         Thread orderThread=new Thread(()->{
10             System.out.println(order);
11         });
12         Thread paymentThread=new Thread(()->{
13             System.out.println(payment);
14             paymentDone=true;
15         });
16         Thread deliveryThread=new Thread(()->{
17             if(paymentDone){
18                 System.out.println(delivery);
19             }else{
20                 System.out.println("Not proceed");
21             }
22         });
23         orderThread.start();
24         orderThread.join();
25         paymentThread.start();
26         paymentThread.join();
27         deliveryThread.start();
28         deliveryThread.join();
29     }
30 }

```

Pizza
UPI
agents

Output

Pizza
UPI
agents

Submitted

Manufacturing Plant -
Producer-Consumer Problem
10 marks

5/5 answered

Your Code

```

1 class Buffer{
2     String item;
3     synchronized void
4         produce(String p) throws InterruptedException{
5         while(item !=null)
6             wait();
7     }
8     item=p;
9     System.out.println("Produced: "+p);
10    notify();
11 }
12 synchronized void consume()
13     throws InterruptedException{
14     while(item == null)
15         wait();
16 }
17     System.out.println("Consumed: "+item);
18     item=null;
19     notify();
20 }
21 }
22 public class Main{
23     public static void main(String[]args) throws InterruptedException{
24         Buffer b=new Buffer();
25         Thread producer= new Thread()->{
26             try{
27                 b.produce("P1");
28                 b.produce("P2");
29                 b.produce("P3");
30             }catch(InterruptedException e){
31                 e.printStackTrace();
32             }

```

Input

stdin input

Output

